LIVE CLASS NOTES

Tool calling - the agent's hands

31 Oct 2026 · 16:48–19:28 · 160 min · Rahul Tiwari

Overview

This session focused on introducing participants to the concept of AI agents, covering foundational theory, practical demonstrations, and a hands-on coding exercise. The instructor, referred to as Rahul throughout the session, guided participants through understanding what agents are, their core properties, when to use them, and how to build a basic code review agent using Python and the OpenAI API. The session also addressed environment setup issues for several participants.

Key Concepts or Theories

- **AI Agent Definition:** A system that perceives input from its environment, makes decisions, and takes actions (such as calling APIs or reading files) to achieve a defined goal
- **Five Properties of Agents:** Autonomy, Perception and Feedback Loop, Continuity (memory), Goal-Orientation, and Flexibility
- **Types of Agents:** Rule-based agents, Learning-based agents (ML/DL models), and LLM-powered agents
- **Tool Calling:** The mechanism by which an LLM recommends which Python functions (tools) to execute; the developer writes the execution logic
- **Tool Schema:** A structured dictionary definition that describes each tool's name, purpose, and parameters so the LLM can understand when and how to use it
- **RAG (Retrieval-Augmented Generation):** A pipeline for retrieving relevant document chunks and feeding them to an LLM to answer domain-specific questions
- **Agentic Loop:** The iterative process in which an agent calls tools, receives results, re-feeds them to the LLM, and repeats until the goal is achieved or a maximum iteration limit is reached

Important Questions Raised

- When should an agentic solution be used versus a simpler LLM call?
- How does an LLM know which tool to call, and who actually executes the tool?
- What are the core components of any AI agent?
- How can agents be prevented from entering infinite loops?
- Is data sent via API calls stored or exposed by OpenAI?
- What is the appropriate scope and number of tools for a single agent?

Key Takeaways and Summary of Learning Objectives

- An AI agent perceives input, makes decisions, and uses tools to achieve a defined goal — it does not act entirely on its own without developer-defined logic
- LLMs recommend which tools to call but cannot execute them directly; the developer must write the execution code
- Any agent has three core components: a model (LLM), instructions (prompt), and tools (Python functions)
- Agents should be given a limited, well-defined set of tools and responsibilities; too many tools or tasks in one agent is discouraged
- Agentic solutions are appropriate for complex, multi-source, or unstructured data problems; simple LLM calls suffice for straightforward queries
- A maximum iteration limit should be set to prevent agents from entering infinite loops
- API calls to OpenAI do not retain memory between calls; memory management is the developer's responsibility
- Phase 0 of the code review agent reads files locally; Phase 1 (future session) will integrate directly with GitHub via APIs
- LangGraph will be introduced in the next session to simplify the agent-building process significantly
- Basic Python proficiency (functions, loops, conditionals) is a prerequisite for following the coding portions of the bootcamp

Topic 1: What Is an AI Agent and Its Properties

An AI agent, in the context of artificial intelligence, is a system that takes input from its environment, reasons about what action to take, and either generates a final response or calls external resources (tools) to gather more information before responding. This concept is not new — it has existed in AI research for approximately 70 years — but has gained renewed relevance with the rise of LLM-powered systems.

Rahul outlined five key properties that characterize most agents. First, **autonomy** means agents operate with minimal human intervention, requiring only high-level instructions such as a pull request number to review. Second, **perception and feedback loop** describes how agents continuously read environmental input and assess whether they have enough information to fulfill their goal. Third, **continuity** refers to the agent's ability to maintain memory across steps, enabling multi-turn reasoning. Fourth, **goal-orientation** emphasizes that every agent must have a clearly defined objective; without one, the agent is effectively useless. Fifth, **flexibility** means agents can adapt their approach mid-task if conditions change, while still pursuing the same overarching goal.

Rahul also distinguished three types of agents: rule-based agents (simple if-else logic, no LLM required), learning-based agents (trained ML/DL models that learn from historical data), and LLM-powered agents (the primary focus of this bootcamp), where a large language model interprets natural language input and decides what actions to take.

Relevant Q&A

Question: What is the criteria or condition for an agent to decide which action to take when given stock names?

Answer: The agent reads the goal defined by the developer (e.g., compare two stocks and recommend buy or sell). It recognizes it needs real-time data, recommends calling a predefined tool (such as a financial data API), retrieves the data, re-feeds it to itself, and then generates a final recommendation once it determines it has sufficient information.

Question (Abhinav Singh): If the fifth property (flexibility) is not implemented initially, will the agent still work, and will it create limitations later?

Answer: It depends on the use case. If the goal is fixed and instructions are predefined, the agent can still function. Runtime instruction changes require additional engineering. This topic will be revisited when tool calling is covered in depth.

Topic 2: When to Use an Agentic Solution

Not every problem requires an agentic solution. Rahul demonstrated this using a real-world example: a company's internal PDF policy document (Bajaj Finance loan policies). When a user asks ChatGPT for the exact interest rate for a personal loan from that company, ChatGPT returns generic information because it cannot access private, non-public documents.

To solve this, a RAG pipeline can be used — chunking the PDF, vectorizing the chunks, storing them in a vector database, and retrieving relevant chunks to feed to the LLM. However, a purely static RAG pipeline is insufficient because users may ask general questions (e.g., "Hi") that do not require document retrieval. This is where an agentic approach becomes necessary: a router agent classifies incoming questions and decides whether to search the document store or respond directly using the LLM.

The decision to use agents becomes even clearer when multiple data sources are involved (PDFs, databases, PowerPoint files, OneDrive, etc.), when the number of routing conditions becomes too large to manage with simple if-else logic, or when large volumes of unstructured data are present.

Relevant Q&A

Question (Navjot Randhawa): Can ChatGPT answer questions from a company's internal PDF?

Answer: No. ChatGPT can only access publicly available information. Internal documents that are not publicly hosted cannot be read by ChatGPT unless explicitly uploaded. For private data, a custom solution with RAG or an agentic pipeline is required.

Question (Raman Aggarwal): Can we use RAG for this?

Answer: Yes, RAG is appropriate here and can function as a component within an agentic pipeline. The agent decides when to invoke the RAG retrieval tool versus when to respond directly.

Topic 3: Core Components of an AI Agent and Tool Calling

Rahul defined the three core components of any AI agent as: the **model** (LLM), the **instruction** (prompt defining the agent's role and goals), and **tools** (Python functions the agent can recommend calling). Memory is an additional component that will be addressed in later sessions.

The hands-on demonstration built a code review agent in plain Python. The agent was given three tools: `read_files` (reads a Python file from a local directory), `post_review_comment` (saves a review comment for a given file), and a status-setting function (marks the PR as "pass" or "block"). Each tool was defined as a Python function and described using a tool schema — a dictionary specifying the function name, a natural language description, and the expected parameters.

A critical concept demonstrated was that **LLMs cannot execute tools themselves**. When the agent receives input and determines it needs a tool, it returns a recommendation (a `tool_calls` object) with the tool name and arguments. The developer's code must then execute the appropriate Python function, collect the result, and re-feed it to the LLM as part of the conversation history. This loop continues until the agent determines no further tool calls are needed and generates a final response.

To prevent infinite loops, a `max_steps` parameter was introduced. The `run_agent` function iterates through the agentic loop up to a defined maximum number of steps, after which it returns a message indicating the agent could not complete the task within the allowed iterations.

Relevant Q&A

Question (Mubin Shaikh): Do we need to specify which tool to use in the tools list?

Answer: No. The tool schema descriptions guide the LLM to select the appropriate tool automatically based on the input. The developer does not need to hardcode which tool to call for each situation.

Question: Is there a thumb rule for how many tools one agent should have?

Answer: There is no fixed rule, but best practice is to avoid giving too many tools or too many responsibilities to a single agent. In production-grade systems, responsibilities are split across multiple specialized agents, each with a limited set of tools.

Question (Raman Aggarwal): Can the same flow be used with Bitbucket instead of GitHub?

Answer: Yes. The agent flow remains the same. Only the tool functions need to be updated to use Bitbucket APIs instead of GitHub APIs. The agent architecture and logic are repository-agnostic.

Question (Abhinav Singh): Can this be done using Node.js?

Answer: Yes, but OpenAI does not have an official Node.js SDK. LangChain provides Node.js support. Learning LangChain is recommended for those who prefer JavaScript/TypeScript.

Topic 4: Data Security and Memory in Agent Applications

A participant raised a concern about whether using an API key to send data to OpenAI exposes sensitive information. Rahul clarified that API calls to OpenAI are stateless — the model does not retain memory between individual API calls. This was demonstrated live by asking the LLM a question and then asking it to recall the previous question; it could not, because no memory was maintained.

In contrast, ChatGPT as an application stores conversation history in its own database, which is why it appears to remember context. When building custom applications, memory management is entirely the developer's responsibility. In production environments, conversation history is typically stored in a developer-controlled database (e.g., PostgreSQL), keeping all data within the organization's own infrastructure and ensuring security.

Relevant Q&A

Question (Mohd Danish): By using an API key, are we exposing our data to OpenAI?

Answer: No. API calls are stateless and do not persist between calls. OpenAI does not store the content of API requests in the same way ChatGPT stores chat history. Each call is an independent HTTP request.

Question (Purusottam Anand): If memory is added in a production system, is the data still secure?

Answer: Yes. In production, the memory (conversation history) is stored in the developer's own database on their own infrastructure. It does not reside with OpenAI, making it fully secure and under the organization's control.

Actionable Next Steps / Assignments

- Practice executing the code published to the GitHub repository; clone the repository, navigate to the correct session folder, and run the agent code independently
- Review the `dummy_agent` notebook and experiment with adding a new tool (e.g., a Google search tool) following the same tool schema pattern demonstrated in the session
- Ensure the local development environment is fully set up: Visual Studio Code, Git, Python virtual environment, and all dependencies from `requirements.txt` installed
- Participants who are new to Python should focus on learning Python functions, for loops, while loops, and if-else statements before the next session
- Attend the next session (Tuesday) 15 minutes early if setup issues remain unresolved
- Attempt the problem statements attached to Session 1 and Session 2 in the repository; unresolved items will be discussed together in the next session

---

Supplemental Resources

- **GitHub Repository (Rahul's bootcamp repo):** Contains all session code, IPython notebooks, research papers, design pattern references, and agentic AI session materials organized by session number. Participants can clone this repository to access and execute all code covered in the sessions.
- **GitHub Repository (Industrial-grade agent project):** A separate repository shared by Rahul containing a production-grade multi-agent architecture with defined agent roles, tool limits, and system design diagrams. Useful for understanding real-world agent design patterns.
- **Support Email:** at malkari.com — participants without access to session groups or recordings can contact this address to be routed to the appropriate support team or added to communication groups.
- **Session Recordings:** All live sessions are recorded and available for re-watching. Participants who miss sessions or need to review content are encouraged to watch recordings at 2x speed alongside the repository code.